

단축평가, 객체분해,
문자열조작

단축 평가

(Short-circuit Evaluation)

```
console.log(true && 'hello'); // "hello"  
console.log(false && 'hello'); // false
```

```
console.log(true || 'hello'); // true  
console.log(false || 'hello'); // "hello"
```

단축 평가

// 사용자가 이름을 입력하지 않았을 때(null 또는 undefined) 기본값 할당

```
let userName = '' ; // Falsy
```

```
let displayName = userName || 'Guest' ;
```

```
console.log(displayName); // "Guest"
```

// 값이 존재할 경우

```
userName = 'Wizard' ;
```

```
displayName = userName || 'Guest' ;
```

```
console.log(displayName); // "Wizard"
```

단축 평가

```
let isLoggedIn = true;
let userProfile = { name: 'Alice' };

// 사용자가 로그인했을 때만 이름을 출력
isLoggedIn && console.log(userProfile.name); // "Alice"

// 로그인하지 않았다면 아무 일도 일어나지 않음
isLoggedIn = false;
isLoggedIn && console.log(userProfile.name); // (출력 없음)
```

Null 병합 연산자-??

```
const name1 = null ?? 'Guest';  
console.log(name1);
```

```
const name2 = undefined ?? "Guest";  
console.log(name2);
```

```
const name3 = "RM" ?? "Guest";  
console.log(name3);
```

```
const name4 = "" || "Guest";  
console.log(name4);
```

```
const name5 = "" ?? "Guest";  
console.log(name5);
```

객체 분해(Object Destructuring)

```
const user = {  
  name: 'RM',  
  age: 30,  
};  
// 기존 방식  
const name = user.name;  
const age = user.age;  
  
// 객체 분해 방식  
const { name1, age1 } = user;  
  
console.log(name1);  
console.log(age1);
```

객체 분해 - alias

```
const user1 = {  
  name: 'RM',  
  age: 30,  
};
```

```
const { name: userName, age: userAge } = user1;
```

```
console.log(userName);  
console.log(userAge);
```


객체 분해-객체 매개 변수

```
function userInfo(user) {  
    console.log(`${user.name} - ${user.age}`);  
}
```

```
userInfo({ name: 'RM', age: 30 });
```

```
function printUser({ name, age }) {  
    console.log(`${name} - ${age}`);  
}
```

```
printUser({ name: 'RM', age: 30 });
```


Spread 연산자 ...

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1];  
console.log(arr2);
```

```
const person = { name: 'RM', age: 20 };  
const copy = { ...person };  
console.log(copy);
```

```
const user = { name: '손흥민', age: 20 };  
const newUser = { ...user, age: 21 };  
console.log(newUser);
```

함수 정의-Rest 매개변수

```
function sum(...nums) {  
    console.log(nums);  
}  
sum(1, 2, 3, 4, 5);
```

```
function sum1(num1, num2, ...nums) {  
    console.log(num1);  
    console.log(num2);  
    console.log(nums);  
}  
sum1(1, 2, 3, 4, 5);
```

Rest 매개변수는 반드시 마지막에 와야 한다

문자열 조작

```
const str = 'JavaScript';
```

```
console.log(str.length);  
console.log(str.toUpperCase());  
console.log(str.toLowerCase());  
console.log(str.includes('script'));  
console.log(str.includes('Java'));  
console.log(str.startsWith('java'));  
console.log(str.endsWith('Script'));  
console.log(str.indexOf('a'));  
console.log(str.indexOf('z'));  
console.log(str.slice(0, 4));  
console.log(str.slice(4));  
console.log(str.replace('Script', 'script'));
```

문자열 조작

```
const str1 = 'a,b,c';  
const arr = str1.split(',');  
console.log(arr);  
const str2 = '  hello  ';  
console.log(str2.trim());  
const str3 = 'ha';  
console.log(str.repeat(3));  
const str4 = 'hello';  
console.log(str4.charAt(1));
```

예외처리

try-catch-finally

```
try {  
    // 실행할 코드  
} catch (error) {  
    // 에러 발생 시 실행  
}
```

```
try {  
    console.log(a); // a는 정의되지 않음  
} catch (error) {  
    console.log('에러 발생!');  
}
```

```
try {  
    console.log('실행');  
} catch (e) {  
    console.log('에러');  
} finally {  
    console.log('무조건 실행');  
}
```

에러 throw

```
try {  
    throw new Error('문제 발생');  
} catch (error) {  
    console.log(error.message);  
}  
  
function checkAge(age) {  
    if (age < 18) {  
        throw new Error('미성년자');  
    }  
    return '통과';  
}  
  
try {  
    checkAge(15);  
} catch (error) {  
    console.error(error);  
}
```


Javascript Module

Javascript Module

- 모듈이란 코드를 파일 단위로 나누고 필요한 것만 가져다 쓰는 구조

- 코드 분리 (유지보수)
- 재 사용성 증가
- 협업 용이

- import 방식(ES Module)

- ES6 이후 Javascript 표준 방식

- require 방식 (CommonJS)

- Javascript를 브라우저 외부에서 사용 시 모듈화 작업을 할수 있게 CommonJS라는 워킹 그룹이 제안한 방식
- Node.js는 모듈 명세 1.0을 준수, Node.js 사용 시 기본

named export/import

```
//math.js
```

```
export const add = (a, b) => a + b;
```

```
export const multiply = (a, b) => a * b;
```

```
// export { add, multiply };
```

```
//app.js
```

```
import { add, multiply } from './math.js';
```

```
console.log(add(2, 3)); // 5
```

```
console.log(multiply(2, 3)); // 6
```

default export/import

//greet.js

```
export default function greet(name) {  
  return `${name}님, 안녕하세요.`;  
}
```

//app.js

```
import greet from './greet.js';  
  
console.log(greet('로키'));
```

named+default

```
//user.js
```

```
export const age = 20;
```

```
export default function getName() {  
  return '그레이스';  
}
```

```
//app.js
```

```
import getName, { age } from './user.js';
```

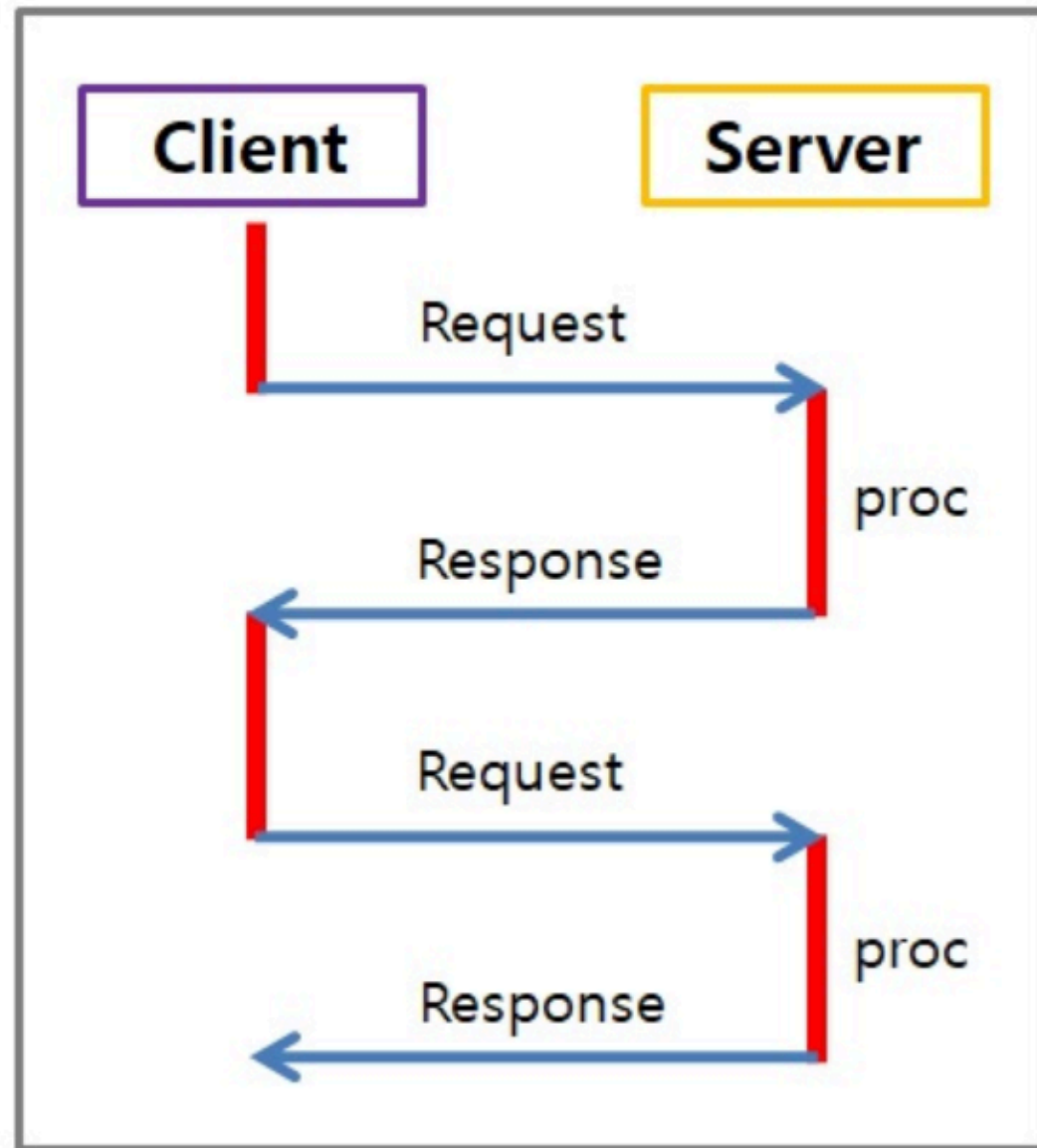
```
console.log(getName()); // YR
```

```
console.log(age); // 20
```

비동기 프로그래밍

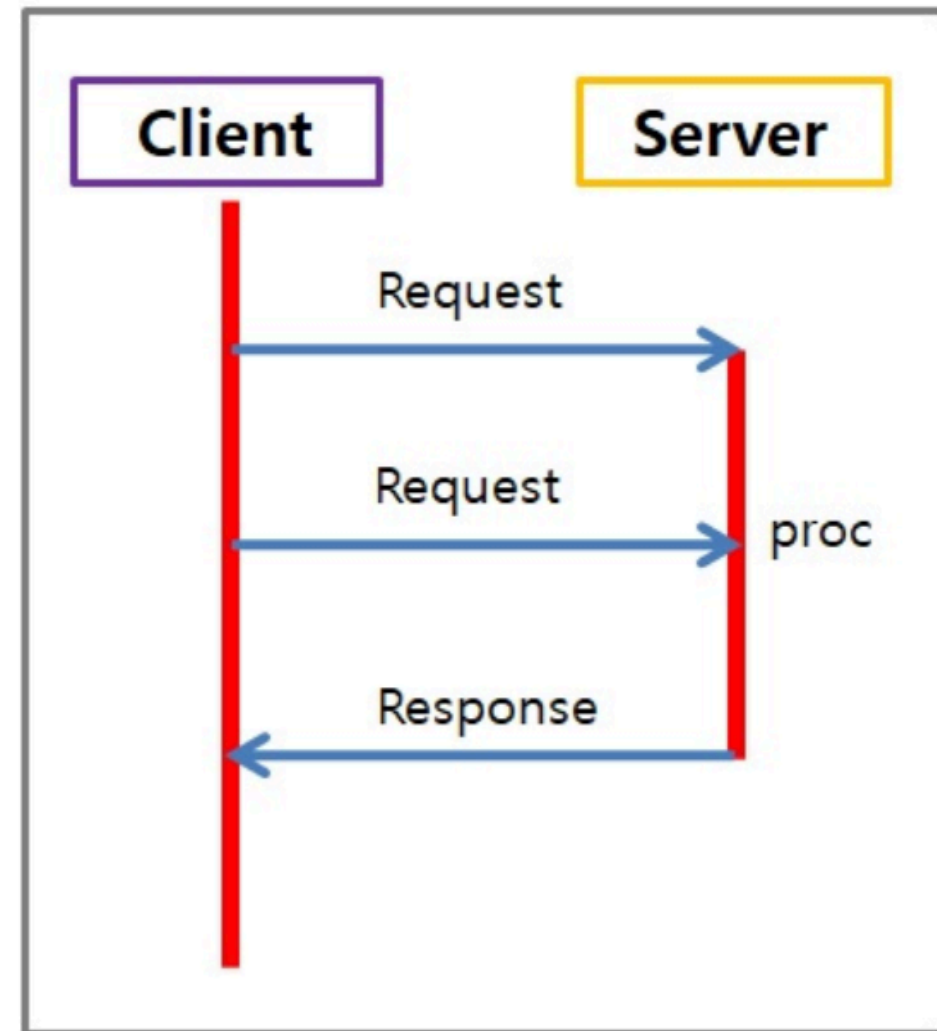
동기 프로그래밍

```
console.log('1번');  
console.log('2번');  
console.log('3번');
```



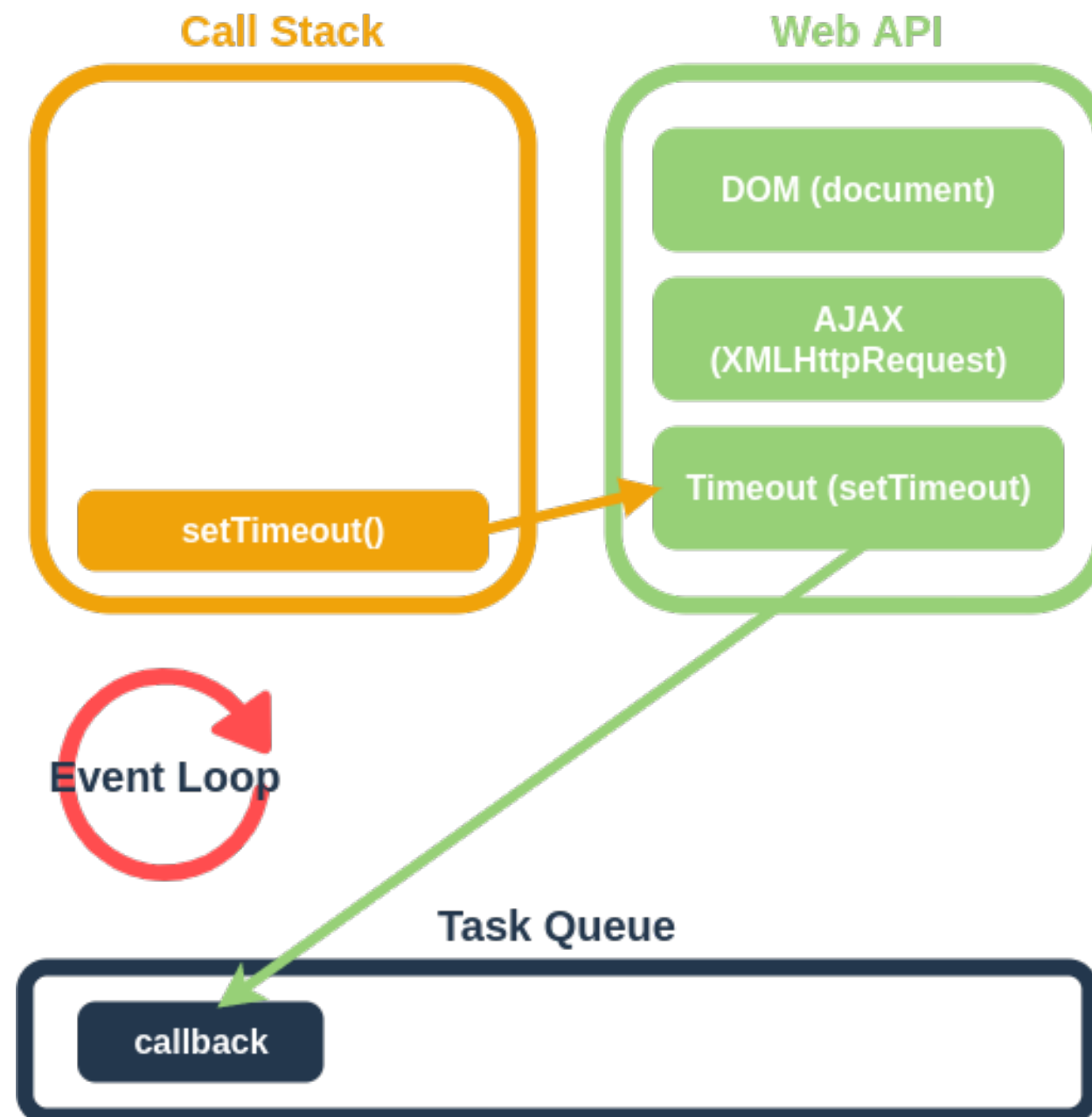
비동기 프로그래밍

```
console.log('1번');  
setTimeout(() => {  
  console.log('2번');  
}, 1000);  
console.log('3번');
```



- setTimeout
 - 특정 작업을 일정 시간이 지난 후에 실행하도록 예약하는 비동기 함수
 - n 밀리초(ms) 후에 전달된 콜백을 실행 시킵니다.

비동기 프로그래밍-Web API (Event Loop, Task Queue)



비동기 프로그래밍

```
function asyncTest(name, callback) {  
  console.log('타이머 시작');  
  setTimeout(() => {  
    callback(name);  
  }, 3000); // 3초 후에 데이터를 가져온다고 가정  
}
```

```
function doOtherthing() {  
  for (let i = 0; i < 300; i++) {  
    console.log(`${i}번째 처리`);  
  }  
}  
  
asyncTest('뷔', sayHello);  
doOtherthing();
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
const DB = [];
```

```
function save2DB(user, callback) {  
  DB.push(user);  
  console.log(`${user.name}님 데이터베이스에 저장 완료되었습니다.`);  
  return callback(user);  
}
```

```
function sendEmail(user, callback) {  
  console.log(`${user.email}으로 이메일이 전송 완료되었습니다.`);  
  return callback(user);  
}
```

```
function getResult(user) {  
  return `${user.name}님 회원가입에 성공했습니다.`;  
}
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
function register(user) {  
  return save2DB(user, (user) => {  
    return sendEmail(user, (user) => {  
      return getResult(user);  
    });  
  });  
}
```

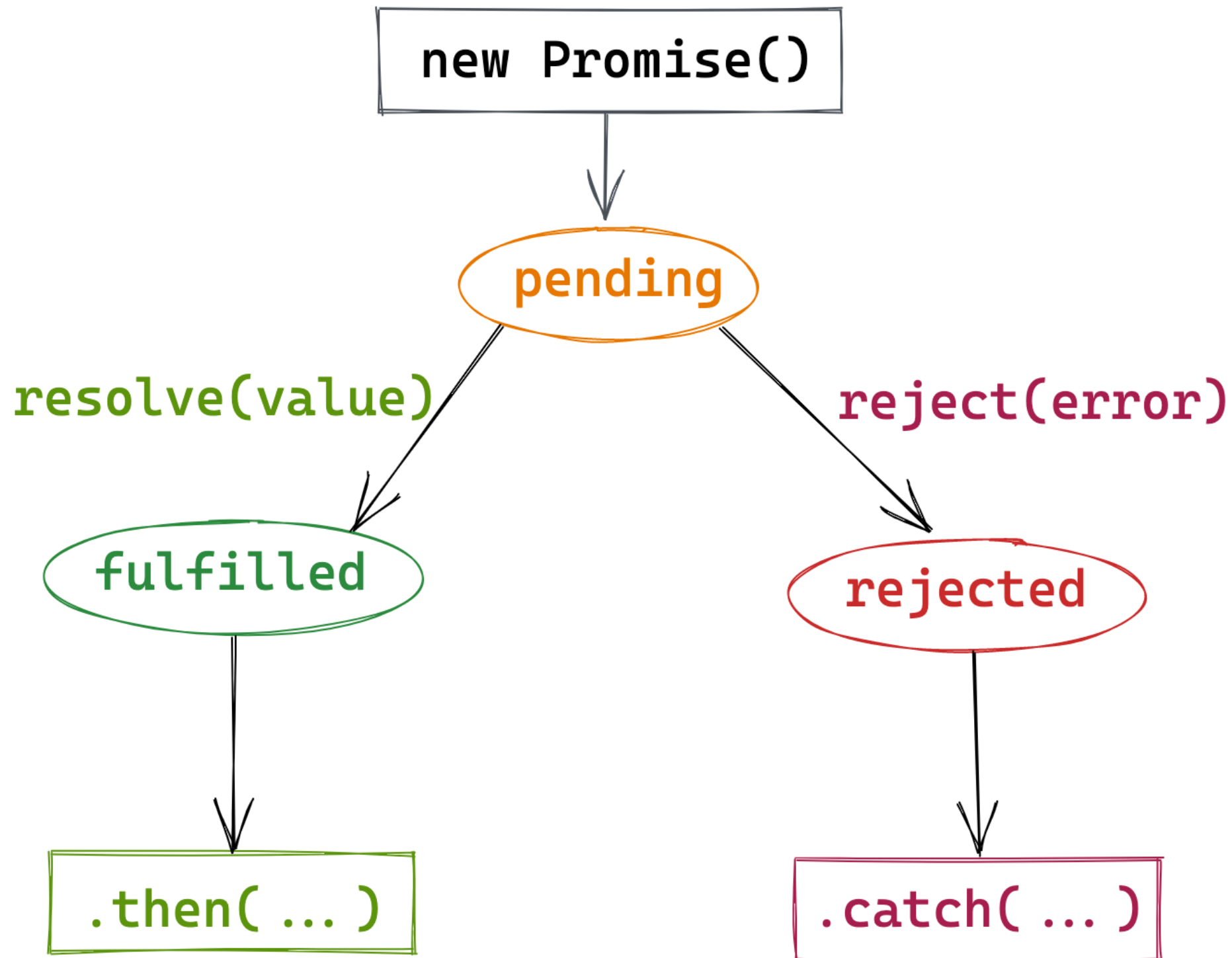


```
const result =  
  register({ name: '손흥민', email: 'son@naver.com' });  
console.log(result);
```

Promise

- 콜백의 단점을 보완하기 위해 만들어진 비동기 처리 객체
- 비동기 처리 시점을 명확히 표현 가능
- 연속된 비동기 처리 작업을 처리하기 간편
- Promise는 3개의 상태를 가짐
 - pending: 아직 실행되지 않은 초기 상태
 - fulfilled: 연산이 성공적으로 완료됨
 - rejected: 연산이 실패함

Promise



Promise

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve('작업 성공!');  
  } else {  
    reject('작업 실패!');  
  }  
});  
promise  
  .then((result) => {  
    console.log('성공 결과:', result);  
  })  
  .catch((error) => {  
    console.error('실패 결과:', error);  
  });
```

Promise

```
const p = new Promise((resolve) => {  
  console.log('1. Promise 실행');  
  resolve(10);  
});
```

```
console.log('2. 코드 계속 실행');
```

```
p.then((num) => {  
  console.log('3. then 실행:', num);  
});
```


Promise-체이닝

```
const p1 = new Promise((resolve) => {  
  const result = 10;  
  resolve(result);  
});
```

//반환값:11 => resolve(11)을 호출하는 Promise를 생성

```
const p2 = p1.then((num) => num + 1);
```

```
p2.then((num) => console.log(num));
```

```
p1.then((num) => num + 1).then((num) => console.log(num));
```


즉시 성공 Promise

```
new Promise((resolve) => {  
    resolve(10);  
});  
Promise.resolve(10);  
//함수가 항상 Promise를 반환하게 만들기  
function getData() {  
    return Promise.resolve('데이터');  
}  
getData().then(console.log);  
//Promise 체이닝을 사용  
Promise.resolve(1)  
    .then((n) => n + 1)  
    .then(console.log);
```

즉시 실패 Promise

```
Promise.reject('에러').catch(console.log);
```

```
function checkAge(age) {  
  if (age < 18) {  
    return Promise.reject('미성년자');  
  }  
  return Promise.resolve('통과');  
}
```

```
checkAge(15).catch(console.log);
```

Promise - 회원가입

```
const DB = [];  
  
function saveDB(user) {  
  const oldDBLength = DB.length;  
  DB.push(user);  
  console.log(`${user.username} 저장 완료되었습니다.`);  
  return new Promise((resolve, reject) => {  
    if (DB.length > oldDBLength) {  
      resolve(user);  
    } else {  
      reject(new Error('저장에 실패했습니다.!' ));  
    }  
  });  
}
```

Promise - 회원가입

```
function sendEmail(user) {  
  console.log(`${user.email}으로 이메일을 발송했습니다.`);  
  return new Promise((resolve) => {  
    resolve(user);  
  });  
}
```

```
function getResult(user) {  
  return new Promise((resolve) => {  
    resolve(`${user.username}님 등록 성공했습니다.`);  
  });  
}
```

Promise - 회원가입

```
function registerByPromise(user) {  
  const result = saveDB(user)  
    .then(sendEmail)  
    .then(getResult)  
    .catch((error) => new Error(error));  
  return result;  
}
```

```
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
const result = registerByPromise(myUser);  
result.then(console.log);
```

Promise - 영화정보

```
const url =  
  'http://raw.githubusercontent.com/wapj/jsbackend/main/  
movieinfo.json';  
  
fetch(url)  
  .then((response) => {  
    return response.json();  
  })  
  .catch(() => {  
    console.log('요청에 실패했습니다.');  })
```

Promise - 영화정보

=> 앞에서 계속

```
.then((data) => {  
    if (!data) {  
        throw new Error('데이터가 없습니다.');    }  
    if (!data.articleList || data.articleList.length === 0) {  
        throw new Error('데이터가 없습니다.');    }  
    return data.articleList;  
})  
.catch((error) => {  
    console.error('에러 발생:', error.message);  
})
```


Promise - 영화정보

=> 앞에서 계속

```
.then((articles) => {  
    return articles.map((article, idx) => {  
        return { title: article.title, rank: idx + 1 };  
    });  
})  
.then((results) => {  
    for (let movie of results) {  
        console.log(`[${movie.rank}위] ${movie.title}`);  
    }  
})  
.catch((err) => {  
    console.log('<<에러발생>>');  
    console.log(err);  
});
```


async/await

- Promise를 더 쉽게 다루기 위한 문법
- 비동기 코드를 동기 코드처럼 깔끔하게 작성 가능
- async function은 자동으로 Promise로 감싸서 반환
- await는 Promise가 해결될 때까지 대기
- await는 반드시 async 함수 안에서만 사용 가능
- await 오른쪽은 반드시 Promise

async/await

// async 키워드가 붙은 함수는 반환값을 Promise로 감싸서 반환

```
async function func1() {  
  return 'hello';  
}
```

```
func1().then(console.log);
```

```
function func2() {  
  return new Promise((resolve) => {  
    resolve('hello');  
  });  
}
```

```
func2().then(console.log);
```

async/await

```
async function func3() {  
  //await는 뒤에 오는 프로미스가 실행완료될때까지 기다림  
  //async함수에서만 사용가능  
  let name = await func1();  
  console.log(name);  
}  
  
func3();
```

async/await

```
function waitOneSecond(msg) {  
  return new Promise((resolve, _) => {  
    setTimeout(() => resolve(`${msg}`), 1000);  
  });  
}
```

```
async function countOneToTen() {  
  for (let x of [...Array(10).keys()]) {  
    let result = await waitOneSecond(`${x + 1}초 대기 중...`);  
    console.log(result);  
  }  
  console.log('완료');  
}
```

```
countOneToTen();
```

async/await - 회원가입

```
async function registerByAsync(user) {  
  try {  
    const savedUser = await saveDB(user);  
    const emailedUser = await sendEmail(savedUser);  
    const result = await getResult(emailedUser);  
    return result;  
  } catch (error) {  
    return new Error(error);  
  }  
}  
  
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
registerByAsync(myUser).then(console.log);
```

async/await - 영화정보

// 1. 데이터 가져오기(fetch)

```
async function fetchMovieData(url) {  
  const response = await fetch(url);  
  if (!response.ok) {  
    throw new Error('요청에 실패. 상태 코드: ' + response.status);  
  }  
  const data = await response.json();  
  return data;  
}
```

// 2. 데이터 검증

```
function validateMovieData(data) {  
  if (!data.articleList || data.articleList.length === 0) {  
    throw new Error('데이터가 없습니다. ');  
  }  
}
```

async/await - 영화정보

// 3. 영화 정보 추출

```
function extractMovieInfos(articleList) {  
  return articleList.map((article, idx) => ({  
    title: article.title,  
    rank: idx + 1,  
  }));  
}
```

// 4. 영화 출력

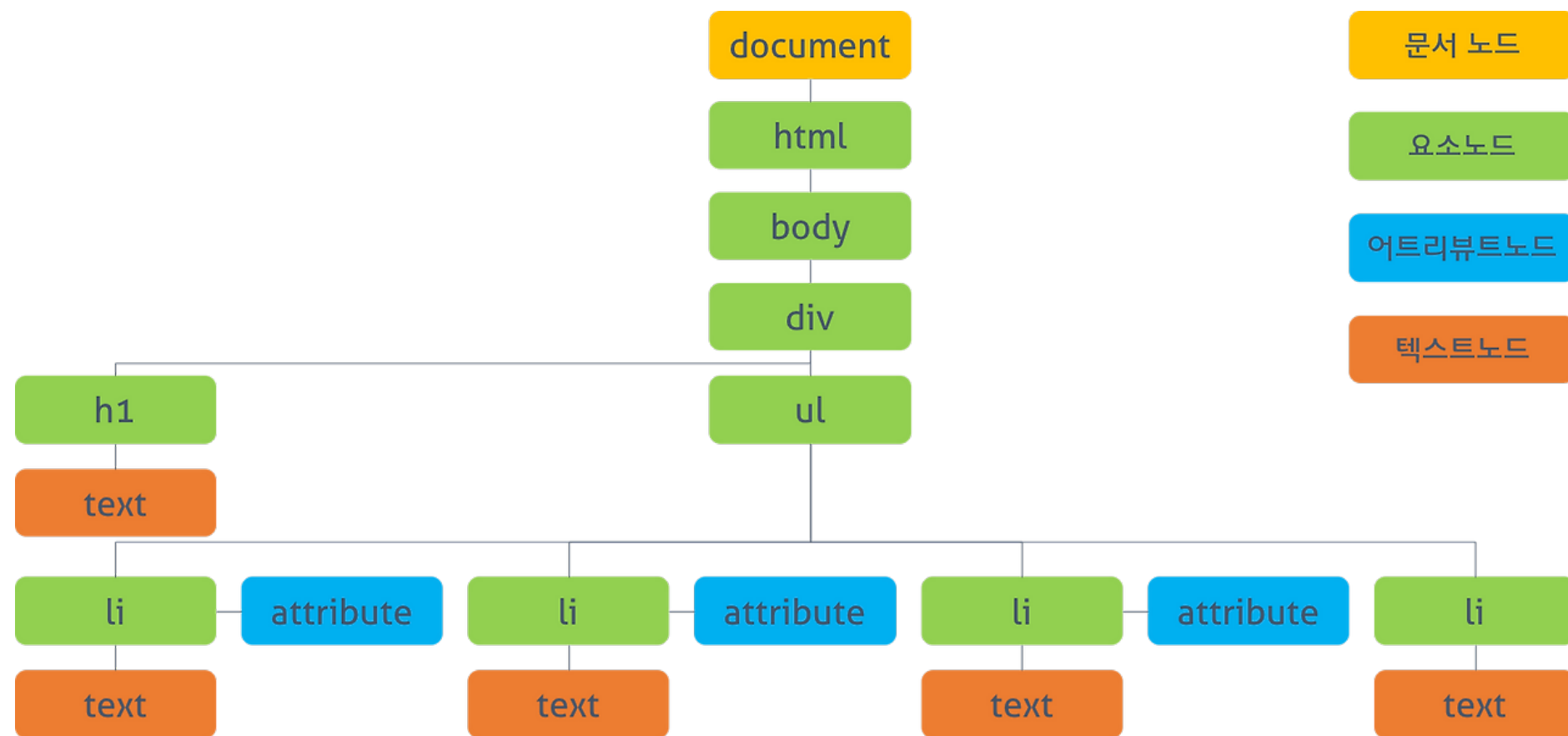
```
function displayMovies(movieInfos) {  
  for (const movie of movieInfos) {  
    console.log(`[${movie.rank}위] ${movie.title}`);  
  }  
}
```


async/await - 영화정보

```
const url = 'http://raw.githubusercontent.com/wapj/  
jsbackend/main/movieinfo.json';  
  
async function movies() {  
  try {  
    const data = await fetchMovieData(url);  
    validateMovieData(data);  
    const movieInfos = extractMovieInfos(data.articleList);  
    displayMovies(movieInfos);  
  } catch (err) {  
    console.error('에러 발생:', err.message);  
  }  
}  
  
movies();
```


DOM(문서 객체 모듈) 다루기

DOM(Document Object Model)



- DOM은 HTML 문서를 객체로 표현한 트리 구조 모델
- 브라우저는 DOM을 사용해 렌더링
- 자바스크립트는 이를 조작한다

요소 선택

```
<body>
  <h1 id="title">제목입니다</h1>
  <div class="box">box 클래스 요소</div>
  <div><p>div 바로 아래에 있는 p 태그</p></div>
  <script>
    const el1 = document.querySelector('#title');
    const el2 = document.querySelector('.box');
    const el3 = document.querySelector('div > p');
    console.log(el1);
    console.log(el2);
    console.log(el3);
  </script>
</body>
```

요소 조작 및 이벤트

```
<body>
  <h1 id="title">안녕하세요</h1>
  <button class="btn">변경</button>
  <script>
    const title = document.querySelector('#title');
    const btn = document.querySelector('.btn');
    btn.addEventListener('click', () => {
      title.textContent = 'querySelector 성공!';
    });
  </script>
</body>
```

다중 선택

```
<body>
  <ul>
    <li>1번</li>
    <li>2번</li>
    <li>3번</li>
  </ul>

  <script>
    const items = document.querySelectorAll('li');
    items.forEach((item) => {
      item.style.color = 'blue';
    });
  </script>
</body>
```

class 조작

```
<body>
  <div class="box">클릭</div>
  <style>
    .active {
      background: yellow;
    }
  </style>
  <script>
    const box = document.querySelector('.box');
    box.addEventListener('click', () => {
      box.classList.toggle('active');
    });
  </script>
</body>
```

요소 생성

```
<body>
  <ul class="list"></ul>
  <button class="add">추가</button>
  <script>
    const list = document.querySelector('.list');
    const btn = document.querySelector('.add');
    btn.addEventListener('click', () => {
      const li = document.createElement('li');
      li.textContent = '새 항목';
      list.appendChild(li);
    });
  </script>
</body>
```


입력처리

```
<body>
  <input class="input" placeholder="입력" />
  <button class="add">추가</button>
  <ul class="list"></ul>
  <script>
    const input = document.querySelector('.input');
    const btn = document.querySelector('.add');
    const list = document.querySelector('.list');
    btn.addEventListener('click', () => {
      const li = document.createElement('li');
      li.textContent = input.value;
      list.appendChild(li);
      input.value = '';
    });
  </script>
</body>
```


DOM 선택 메서드

```
<body>
  <h1 id="title">제목</h1>
  <div class="box">box1</div>
  <div class="box">box2</div>
  <p>문단1</p>
  <p>문단2</p>
  <input type="text" name="username" value="홍길동" />
  <input type="text" name="username" value="김철수" />
  <script>
    const el1 = document.getElementById('title');
    console.log('getElementById:', el1);
    const el2 = document.getElementsByClassName('box');
    console.log('getElementsByClassName:', el2);
    const el3 = document.getElementsByTagName('p');
    console.log('getElementsByTagName:', el3);
    const el4 = document.getElementsByName('username');
    console.log('getElementsByName:', el4);
  </script>
</body>
```

prototype

prototype

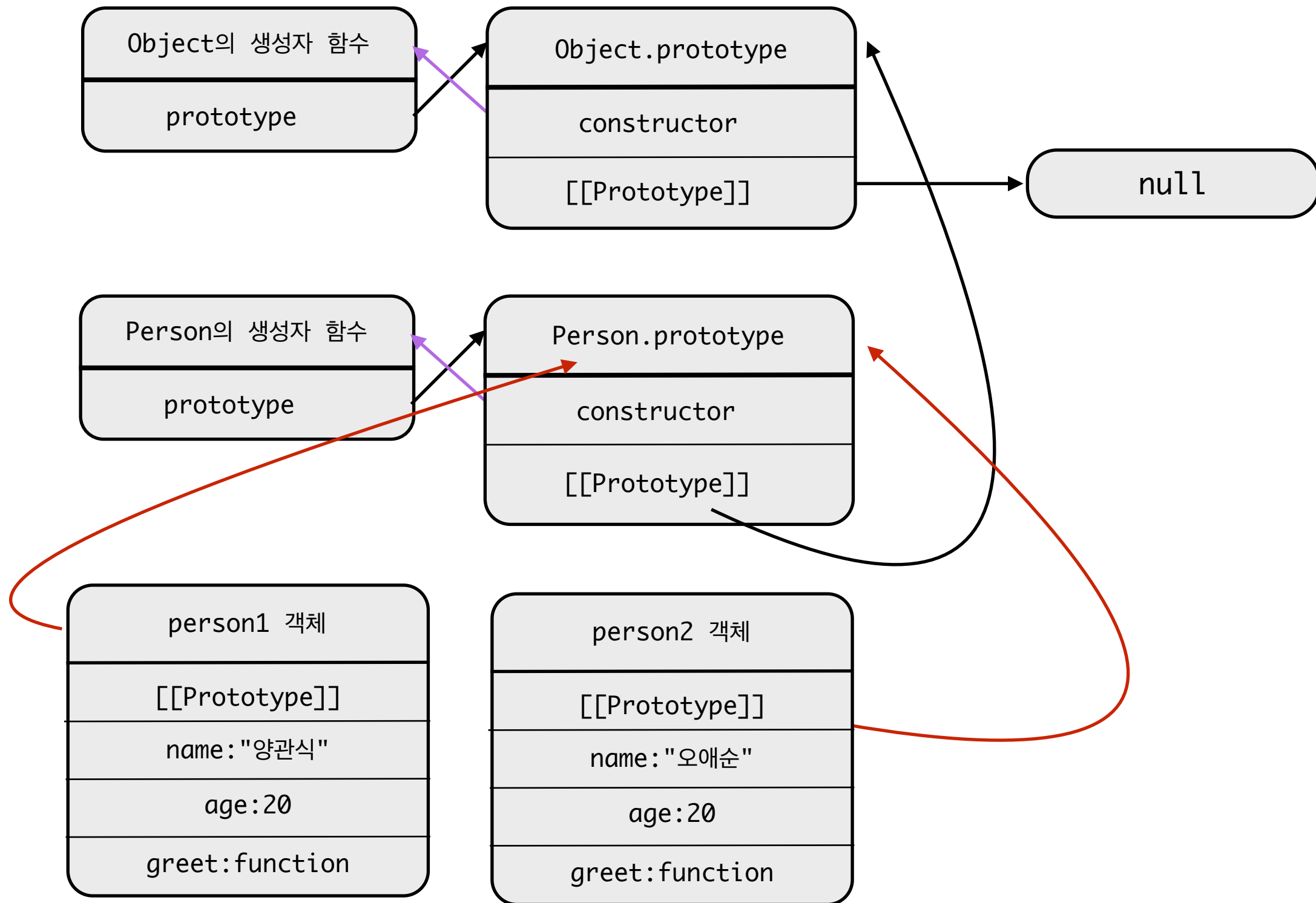
- **prototype은 JavaScript의 상속을 구현하는 핵심 메커니즘**
- **prototype은 객체를 생성할 때 상속될 속성과 메서드를 담는 공간**
- **객체에서 값을 찾을 때 prototype 체인을 따라 올라가며 탐색**
- **객체는 [[Prototype]]를 통해 생성자의 prototype과 연결**
- **모든 생성자 함수는 prototype 객체를 가진다**

생성자 함수

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function () {  
    console.log(`안녕 나는 ${this.name}야!`);  
  };  
}
```

```
const person1 = new Person('양관식', 20);  
const person2 = new Person('오애순', 20);  
console.log(person1.name, person1.age);  
person2.greet();
```

prototype을 이용한 상속



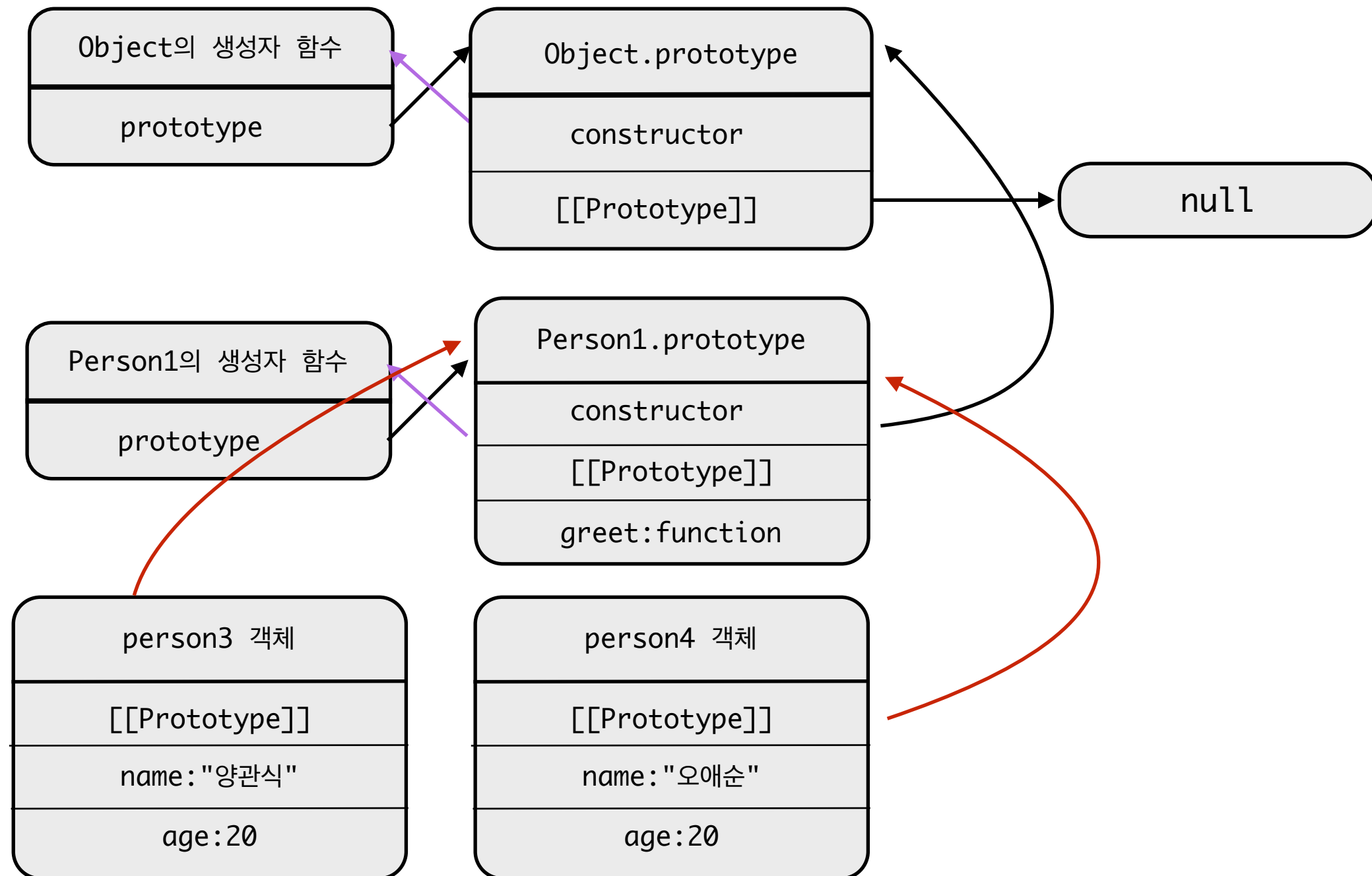
prototype 메서드

```
function Person1(name, age) {  
    this.name = name;  
    this.age = age;  
}  
Person1.prototype.greet = function () {  
    console.log(`안녕 나는 ${this.name}야!`);  
};
```

```
const person1 = new Person1('양관식', 20);  
const person2 = new Person1('오애순', 20);
```

```
console.log(person1.name, person1.age);  
person2.greet();
```

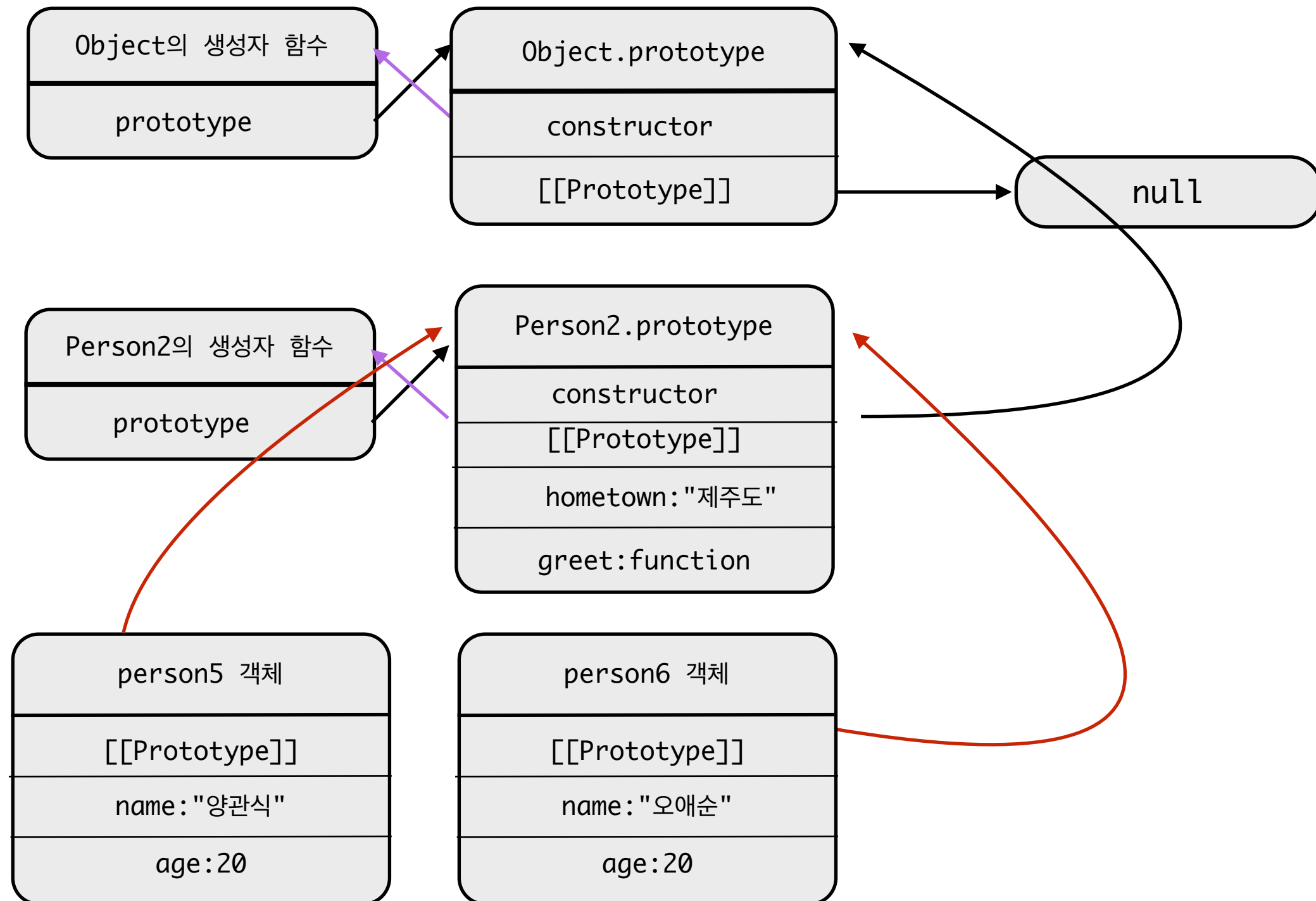
prototype에 메서드 추가



prototype 속성

```
function Person2(name, age) {  
  this.name = name;  
  this.age = age;  
}  
Person2.prototype.greet = function () {  
  console.log(`안녕 나는 ${this.name}야!`);  
};  
Person2.prototype.hometown = '제주도';  
const person5 = new Person2('양관식', 20);  
const person6 = new Person2('오애순', 20);  
console.log(person5.hometown, person6.hometown);  
Person2.prototype.hometown = '서울';  
console.log(person5.hometown, person6.hometown);
```

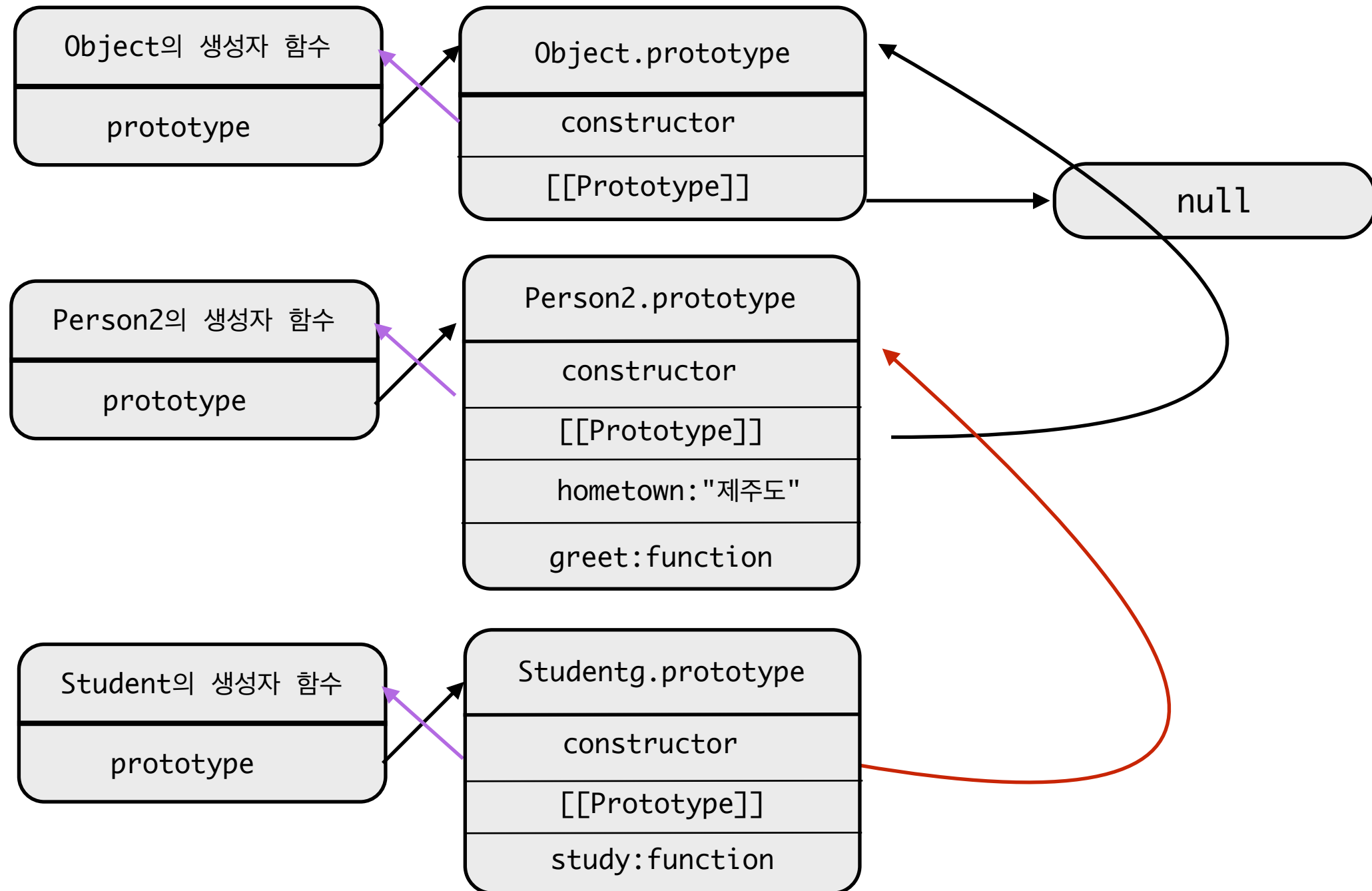

prototype에 속성 추가



prototype를 이용한 상속

```
function Student(name, age, major) {  
    Person2.call(this, name, age); // 부모 생성자 호출  
    this.major = major;  
}  
  
Student.prototype = Object.create(Person2.prototype);  
Student.prototype.constructor = Student;  
Student.prototype.study = function () {  
    console.log(`${this.name}은 ${this.major}를 공부한다`);  
};  
  
const s1 = new Student('양관식', 20, '컴퓨터공학');  
s1.greet(); // 부모 메서드  
s1.study(); // 자식 메서드
```

prototype을 이용한 상속



감사합니다.